ChatGPT

Modern JavaScript Libraries for Browser Games, Graphics, Simulation, CAD/BIM and Scientific Visualisation

Scope and current ecosystem

As of **16 August 2026**, the browser graphics ecosystem is better understood as a stack of complementary layers than as a contest for one universal library. Three.js, Babylon.js and PlayCanvas occupy different points on the general-purpose 3D spectrum; PixiJS and Phaser do the same for 2D rendering versus full game structure; GSAP supplies animation independently of the renderer; Rapier and Matter.js supply physics; D3, Plotly.js and deck.gl address data visualisation at different abstraction levels; CesiumJS and MapLibre GL JS specialise in geospatial rendering; vtk.js specialises in scientific visualisation; xeokit and That Open specialise in AEC/BIM; and OpenCascade.js supplies something fundamentally different again—a real CAD geometry kernel rather than merely a renderer. [1]

The original list in the question remains well chosen. I would **keep all of it**, but add four libraries that materially improve the map:

Library	Layer / abstraction	Best thought of as	Ecosystem position in 2026
Three.js	3D rendering library	General-purpose 3D foundation	Very high adoption; r185 current
Babylon.js	3D rendering/game engine	Batteries-included web 3D engine	Very mature; Babylon.js 9
PlayCanvas	3D game/ application engine	Engine + ECS + optional visual workflow	Mature, active WebGPU/WebGL engine
PixiJS	2D renderer	High-performance scene-graph renderer	Very high 2D adoption; v8 generation
Phaser	2D game framework	Complete browser game framework	Very mature; Phaser 4.2.1
GSAP	Animation system	Renderer-independent tween/timeline engine	De-facto specialist animation tool
Konva	Interactive Canvas framework	Scene graph for editors/ diagrams	Mature and editor-oriented
Fabric.js	Interactive Canvas framework	Rich editable-object canvas	Mature; strong text/ SVG/serialisation
Matter.js	2D physics engine	Pure-JS rigid-body physics	Mature, conservative

Library	Layer / abstraction	Best thought of as	Ecosystem position in 2026
Rapier	2D/3D physics engine	High-performance WASM physics	Strong modern physics choice
D3.js	Data-vis primitives	Low-level bespoke visualisation toolkit	Foundational
Plotly.js	Charting library	High-level scientific/data charts	Strong complement to D3
deck.gl	GPU data-vis framework	Huge geospatial/data layers	Leading specialist
MapLibre GL JS	Web map renderer	Vector-tile basemap/ cartography engine	Major open mapping platform
CesiumJS	3D geospatial engine	Globe, terrain and 3D Tiles renderer	Leading 3D geospatial option
vtk.js	Scientific-vis framework	Surface/volume/medical visualisation	Leading browser VTK-style toolkit
xeokit	BIM/AEC visualisation SDK	Large engineering/BIM model viewer	Mature specialist
That Open / web-ifc	BIM stack + IFC engine	IFC-native open BIM application stack	Rapidly evolving specialist
OpenCascade.js	CAD geometry kernel	B-rep/NURBS/Boolean/ STEP kernel	Important specialist addition

The additions are warranted for distinct reasons. **PlayCanvas** is too substantial a web-first 3D engine to omit when discussing browser games and interactive 3D. **Plotly.js** fills the large gap between D3's primitives and domain-specific scientific rendering. **MapLibre GL JS** is the natural open-source basemap partner to deck.gl and one of the central modern vector-map renderers. **OpenCascade.js** fills the most important conceptual gap of all: Three.js, Babylon.js, Fabric.js and similar libraries draw geometry, whereas OpenCascade.js exposes a genuine CAD modelling kernel derived from Open CASCADE Technology. [2]

Current release activity also supports retaining several libraries that can superficially look “old”. Phaser 4.2.1 was released on **9 July 2026**; Three.js currently identifies itself as **r185**; Fabric.js lists **7.4.0** as its latest release; vtk.js documentation was updated on **4 August 2026**; CesiumJS reached **1.144** in August 2026; and deck.gl's v9 line continues to receive WebGPU-oriented work. [3]

One qualification is important when comparing “popularity”. npm downloads, GitHub stars, commercial deployments and longevity measure different things, so they should not be collapsed into a single league table. As directional examples, recent package data put PixiJS at roughly **819,000 weekly npm downloads** and `@babylonjs/core` at roughly **373,000**, while specialist engineering packages naturally have much smaller public footprints despite substantial professional use. Rapier's main repository showed roughly **5.6k GitHub stars**, xeokit roughly **923**, That Open Components roughly **693**, and OpenCascade.js roughly **916** at the time of research. Those numbers are useful signals of community size, not measures of technical quality or suitability. [4]

General 3D and game engines

Three.js — general-purpose 3D rendering library

Three.js remains the default starting point for many bespoke browser 3D applications. Its abstraction is primarily a **scene graph plus renderer and associated 3D utilities**, rather than a complete game engine. The current official site is on **r185**, and the project exposes both WebGL and increasingly WebGPU-oriented rendering facilities. Its breadth includes cameras, lights, materials, geometry, loaders, animation, post-processing and a large addon ecosystem without imposing an application architecture. [5]

Its greatest strength is precisely this neutrality. A product configurator, engineering viewer, generative-art project, BIM front end, molecular viewer or game can all put their own state-management, physics and UI architecture around Three.js. That is also its main weakness: once an application needs navigation systems, physics, ECS, networking, gameplay state, level tooling and asset management, **you assemble those pieces yourself**. Babylon.js and PlayCanvas are more integrated; vtk.js, CesiumJS and xeokit are much more domain-specific. This makes Three.js particularly strong as the *substrate* beneath a custom application rather than the highest-level solution. [6]

A minimal scene illustrates its relatively low-level model:

```
import * as THREE from 'three';

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(
  60,
  innerWidth / innerHeight,
  0.1,
  100
);
camera.position.z = 3;

const cube = new THREE.Mesh(
  new THREE.BoxGeometry(),
  new THREE.MeshNormalMaterial()
);
scene.add(cube);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
document.body.append(renderer.domElement);

renderer.setAnimationLoop(() => {
  cube.rotation.y += 0.01;
  renderer.render(scene, camera);
});
```

Babylon.js — integrated 3D rendering and game engine

Babylon.js deliberately sits higher in the stack. Its official site describes Babylon.js 9.0 as a web rendering engine, while the broader API encompasses rendering, cameras, materials, particles, animation, post-processing, asset loading, input, GUI, XR, physics integrations and substantial tooling. Babylon also has strong WebGPU support and a visual Playground/Node Material Editor workflow. [7]

Compared with Three.js, Babylon's advantage is **coherence and built-in systems**. For a web-native 3D game, training simulator, digital twin or XR application, more of the expected engine infrastructure already exists under one API. Its trade-off is a larger conceptual and framework surface: an application adopts more of "the Babylon way". For highly bespoke engineering products, that can be less attractive than building narrowly around Three.js. For game-like applications, it can save substantial integration work. Babylon 9 also includes facilities aimed at large-coordinate worlds, illustrating how far it has moved beyond being merely a mesh renderer. [8]

```
import * as BABYLON from '@babylonjs/core';

const engine = new BABYLON.Engine(canvas, true);
const scene = new BABYLON.Scene(engine);

const camera = new BABYLON.ArcRotateCamera(
  'camera',
  0,
  1.2,
  5,
  BABYLON.Vector3.Zero(),
  scene
);
camera.attachControl(canvas, true);

new BABYLON.HemisphericLight(
  'light',
  new BABYLON.Vector3(0, 1, 0),
  scene
);

BABYLON.MeshBuilder.CreateBox('box', {}, scene);

engine.runRenderLoop(() => scene.render());
```

PlayCanvas — web-first 3D engine with ECS

PlayCanvas deserves inclusion alongside Three.js and Babylon.js rather than as a footnote. It is an open-source WebGL/WebGPU engine aimed at games, interactive 3D, AR/VR and visual applications. Its architecture is explicitly **entity-component-system based**, and it offers both direct engine development and a wider editor/tool ecosystem. Current engine documentation exposes cameras, render components, animation, input, rigid-body/collision integration, GLB assets and Gaussian-splat rendering, while the engine supports dual WebGL/WebGPU backends. [9]

The differentiator is workflow. Three.js is usually "bring your own application/editor architecture"; PlayCanvas is closer to a conventional game engine transplanted into web technology. Babylon occupies a similar high-level territory but tends to be more code/API-centric, whereas PlayCanvas'

editor heritage is a strong attraction for teams containing artists and level designers. A current v2.21.2 release in late July 2026 and continuing issue activity indicate an actively maintained platform. [10]

```
import * as pc from 'playcanvas';

const app = new pc.Application(canvas);
app.start();

const camera = new pc.Entity('camera');
camera.addComponent('camera');
camera.setPosition(0, 0, 3);
app.root.addChild(camera);

const box = new pc.Entity('box');
box.addComponent('render', { type: 'box' });
app.root.addChild(box);

app.on('update', dt => {
  box.rotate(10 * dt, 20 * dt, 0);
});
```

For **3D games**, the practical distinction is therefore:

Three.js gives maximum architectural freedom; **Babylon.js** gives the broadest code-centric integrated engine; **PlayCanvas** gives an especially strong engine/editor/ECS workflow. None is simply “faster Three.js”; they differ primarily in how much architecture they take responsibility for. [11]

2D graphics, games and animation

PixiJS — high-performance 2D renderer

PixiJS is fundamentally a **2D rendering engine**, not a complete game framework. Its current v8 generation targets WebGL and WebGPU and provides a scene hierarchy of containers, sprites, text, vector-like graphics, assets, filters and interaction facilities. The project positions itself for games, interactive applications, advertising, education and other rich 2D experiences. [12]

That distinction from Phaser matters. PixiJS is an excellent choice when you want a fast 2D renderer inside your own architecture, perhaps driven by React state, an ECS, a custom editor or GSAP. Phaser is preferable when you actually want **a game framework** with scenes, game objects, tilemaps and game-oriented lifecycle conventions. PixiJS's current Graphics API is also quite capable for dynamic shapes, although for object-manipulation editors Konva or Fabric usually save more work. [13]

```
import { Application, Graphics } from 'pixi.js';

const app = new Application();
await app.init({ resizeTo: window });
document.body.append(app.canvas);
```

```
const circle = new Graphics()
  .circle(100, 100, 40)
  .fill(0x66ccff);

app.stage.addChild(circle);
```

Phaser — full 2D browser-game framework

Phaser is explicitly an open-source **HTML5 game framework** using WebGL and Canvas rendering. Phaser 4 was released in 2026 with a substantially rebuilt WebGL renderer, and **4.2.1** followed on 9 July 2026. Phaser supplies game-oriented concepts such as scenes, sprites/game objects, input, cameras, animation, tweens, tilemaps, sound and integrations that would otherwise have to be assembled around a pure renderer. [14]

Phaser remains strongly **2D-oriented**; official guidance has historically been explicit that it is not a built-in general 3D engine. That makes it an excellent choice for platformers, puzzle games, top-down games, card/board titles and web arcade experiences, but not the natural starting point for a first-person 3D world. Phaser overlaps PixiJS in rendering tasks, but its abstraction boundary is much higher. [15]

```
import Phaser from 'phaser';

new Phaser.Game({
  type: Phaser.AUTO,
  width: 800,
  height: 450,
  scene: {
    create() {
      this.add.text(20, 20, 'Hello Phaser');
    }
  }
});
```

GSAP — renderer-independent animation and sequencing

GSAP is neither a graphics renderer nor a game engine. It is an **animation/tweening and timeline system** whose central strength is manipulating properties over time. `gsap.to()` can animate arbitrary object properties, and its Timeline abstraction provides explicit sequencing, overlap, labels, playback control and nesting. Its renderer independence is why GSAP combines naturally with HTML/CSS/SVG, Three.js objects, PixiJS objects, Canvas scene graphs and other JavaScript state. [16]

Its strongest use cases are presentation-quality interaction, landing pages, storytelling, product experiences, UI motion and choreographed scene transitions. It is not a replacement for PixiJS, Three.js or Phaser: it controls **how values change over time**, while those libraries determine how those values become pixels. That separation often produces an excellent pairing—particularly **Three.js + GSAP** or **PixiJS + GSAP**. [17]

```
import { gsap } from 'gsap';
```

```
gsap.to('.box', {
  x: 300,
  rotation: 360,
  duration: 1.2,
  ease: 'power2.inOut'
});
```

Konva — interactive Canvas scene graph

Konva wraps the HTML Canvas with an object-oriented scene graph, event system, hit detection, dragging, grouping, transforms and state management. It is designed specifically for **interactive 2D graphics**, including diagrams, whiteboards, editors, floor plans and annotation tools. It has official integrations for React, Vue, Svelte and Angular. [18]

Konva's sweet spot is not maximum sprite throughput; it is making interactive objects easy. A rectangle can simply be `draggable: true`, while transformer and event APIs handle manipulation. Compared with Fabric.js, Konva tends to feel more like a clean scene graph on Canvas and is particularly attractive for diagramming or custom application UIs. Fabric is often stronger when rich text editing, SVG interchange and document-like serialisation are first-class requirements. [19]

```
import Konva from 'konva';

const stage = new Konva.Stage({
  container: 'app',
  width: 600,
  height: 400
});

const layer = new Konva.Layer();
stage.add(layer);

layer.add(new Konva.Rect({
  x: 40,
  y: 40,
  width: 120,
  height: 80,
  fill: 'skyblue',
  draggable: true
}));
```

Fabric.js — editable object model over Canvas

Fabric.js similarly places an interactive object model above Canvas, but its identity is more explicitly that of a **rich editable-canvas/document system**. The current project lists version 7.4.0 and supports object selection and transformation, text editing, SVG parsing/export and JSON serialisation/restoration. [20]

For a design editor, signage editor, image annotator, label designer or diagram tool where users manipulate text, images and vector objects and you need to save the document, Fabric is often the

more direct fit. For very large custom diagrams or an application where you want to control the scene graph more explicitly, Konva may feel lighter conceptually. Neither is a CAD kernel: dimensions, snapping, constraints, topology and precise geometric operations remain application-level concerns unless paired with more specialised geometry software. [21]

```
import { Canvas, Rect } from 'fabric';

const canvas = new Canvas('canvas');

canvas.add(new Rect({
  left: 40,
  top: 40,
  width: 120,
  height: 80,
  fill: 'skyblue'
}));
```

A useful shorthand is therefore:

```
PixiJS  = "draw lots of 2D things efficiently"
Phaser  = "build a 2D game"
GSAP    = "animate properties and timelines"
Konva   = "build an interactive Canvas application/editor"
Fabric  = "build an editable Canvas document/design tool"
```

That taxonomy is an inference from the projects' stated abstraction boundaries and feature sets rather than a claim that any of them is incapable of crossing into another category. [22]

Physics and data visualisation

Matter.js — accessible pure-JavaScript 2D rigid-body physics

Matter.js remains one of the simplest ways to add 2D rigid-body dynamics in JavaScript. Its current documentation is on **0.20.0** and exposes engines, bodies, composites, constraints, collision handling, gravity, sleeping and an optional Canvas renderer. The project's own documentation describes `Matter.Render` primarily as a lightweight visualisation/debug renderer, so production applications commonly use Matter for simulation while rendering through another system. [23]

Its strengths are approachability, pure-JavaScript deployment and a mature 2D API. Its limitations are equally clear: it is not a 3D solver and is not aimed at the same performance/determinism territory as modern WASM engines such as Rapier. Matter is excellent for casual 2D games, educational demos, draggable physical objects and moderate-complexity simulations; Rapier is generally the stronger starting point when demanding physics or 3D is likely. [24]

```
import Matter from 'matter-js';

const engine = Matter.Engine.create();
```

```
Matter.Composite.add(engine.world, [
  Matter.Bodies.rectangle(400, 100, 80, 80),
  Matter.Bodies.rectangle(400, 580, 800, 40, {
    isStatic: true
  })
]);

const render = Matter.Render.create({
  element: document.body,
  engine,
  options: { width: 800, height: 600 }
});

Matter.Render.run(render);
Matter.Runner.run(Matter.Runner.create(), engine);
```

Rapier — modern high-performance 2D/3D physics through WebAssembly

Rapier is a Rust physics engine with JavaScript/WASM packages for both **2D and 3D**. It provides rigid bodies, colliders, joints, scene queries and character-controller facilities, and the JavaScript/WASM build is documented as cross-platform deterministic when the same version and initial conditions are used. [25]

A potentially confusing 2026 ecosystem detail is that the former standalone `rapier.js` repository was archived in July—not because the JavaScript bindings were abandoned, but because they were merged into the main Rapier repository. The main project remains active. This makes Rapier the more future-facing choice of the two physics libraries here when 3D, determinism or higher-performance simulation matters. The cost is a little more deployment/integration complexity because WASM must be initialised and your render objects must be synchronised with physics bodies. [26]

```
import RAPIER from '@dimforge/rapier3d-compat';

await RAPIER.init();

const world = new RAPIER.World({
  x: 0,
  y: -9.81,
  z: 0
});

const body = world.createRigidBody(
  RAPIER.RigidBodyDesc.dynamic()
    .setTranslation(0, 2, 0)
);

world.createCollider(
  RAPIER.ColliderDesc.ball(0.5),
  body
);
```

```
world.step();
console.log(body.translation());
```

A very common 3D architecture is consequently **Three.js + Rapier**: Three owns the visual scene while Rapier owns collision and dynamics. The same separation works with Babylon or another renderer, although Babylon already provides higher-level physics integration facilities. [27]

D3.js — primitives for bespoke data-driven visualisation

D3 remains foundational because it solves a different problem from a chart library. Its official description emphasises a **low-level, web-standards-based approach** for bespoke data visualisation: selections and data joins, scales, axes, shape generators, layouts, transitions and data transformation. Many modules can also be used independently of DOM rendering. [28]

The benefit is almost unlimited customisation. The downside is that D3 often asks you to design the visualisation yourself. It is therefore a poor choice when "I need a normal line/heatmap/surface chart now" is the problem; Plotly.js is higher level for that. Conversely, when the visual itself is novel—network diagrams, bespoke SVG interaction, data storytelling, custom axes or hybrid Canvas/SVG views—D3 is much more flexible. D3's `d3-force` module even contains a particle-force simulation, but this is designed for network/layout visualisation rather than physical-world rigid-body simulation. [29]

```
import * as d3 from 'd3';

const data = [4, 8, 15, 16, 23, 42];

d3.select('#chart')
  .selectAll('div')
  .data(data)
  .join('div')
  .style('width', d => `${d * 6}px`)
  .text(d => d);
```

Plotly.js — declarative charts for scientific and analytical applications

Plotly.js is a high-level declarative charting library built above D3 and other lower-level technology. Its official documentation advertises more than 40 chart types, with statistical charts, heatmaps, contour plots, 3D scatter/surface visualisations and map-based plots. Charts are described primarily through declarative JavaScript/JSON structures. [30]

For engineering dashboards, laboratory results, simulation outputs and scientific applications, it is often a far more productive choice than hand-building conventional graphs in D3. The trade-off is control: once the desired visualisation departs substantially from Plotly's trace/layout model, D3—or a custom WebGL renderer—becomes more attractive. Recent 2026 release activity, including work around the current v3 line and a v4 release candidate, also justifies including Plotly in a "modern" survey rather than treating it as a legacy charting option. [31]

```
import Plotly from 'plotly.js-dist-min';

Plotly.newPlot('plot', [{
  x: [1, 2, 3],
  y: [2, 1, 4],
  type: 'scatter'
}], {
  margin: { t: 20 }
});
```

deck.gl — GPU visualisation of very large datasets

deck.gl is a **GPU-powered data-visualisation framework** built around composable layers. Its current architecture targets WebGPU and WebGL2 and is specifically designed for large datasets, picking/filtering and geographical projections; its standard catalogue includes scatter, arc, heatmap, hexagon, point-cloud, terrain, 3D Tiles and many other layers. [32]

It overlaps with D3 only loosely. D3 is strongest at bespoke visual encoding and DOM/SVG-oriented composition; deck.gl is strongest when millions of data marks need GPU rendering. It also does **not** necessarily replace a map renderer: deck.gl explicitly integrates with MapLibre, Google Maps, Mapbox and Esri, and its MapLibre integration can synchronise cameras precisely. Think “analytical GPU layers” rather than “complete GIS”. [33]

```
import { Deck } from '@deck.gl/core';
import { ScatterplotLayer } from '@deck.gl/layers';

new Deck({
  initialViewState: {
    longitude: 151.21,
    latitude: -33.87,
    zoom: 10
  },
  controller: true,
  layers: [
    new ScatterplotLayer({
      data: [{ position: [151.21, -33.87] }],
      getPosition: d => d.position,
      getRadius: () => 100
    })
  ]
});
```

Engineering, scientific, BIM, CAD and geospatial

This category is where choosing merely “the best 3D library” becomes actively misleading. A VTK volume, an IFC building, a STEP solid and a planet-scale 3D Tiles scene may all appear as triangles on screen eventually, but their data models, precision requirements, interaction semantics and preprocessing pipelines are radically different.

CesiumJS — planet-scale 3D geospatial visualisation

CesiumJS is purpose-built for **3D globes, terrain, maps and large geospatial datasets**. Its design emphasises geographical precision, interoperability and massive datasets, and it is widely used for aerospace, smart-city, drone and geospatial applications. [34]

That specialisation is its main advantage over Three.js. A globe, WGS84 coordinates, terrain, imagery, 3D Tiles, time-varying geographical entities and very large world coordinates are native concerns in Cesium rather than infrastructure you have to invent. Conversely, for a small local product visualiser or conventional game world, Cesium's geospatial abstraction is unnecessary weight. The August 2026 **1.144** release continuing to add capabilities, including CAD-oriented planar-fill work, indicates an actively evolving project. [35]

```
import {
  Viewer,
  Cartesian3
} from 'cesium';

const viewer = new Viewer('cesiumContainer');

viewer.entities.add({
  position: Cartesian3.fromDegrees(151.21, -33.87),
  point: { pixelSize: 10 }
});

viewer.zoomTo(viewer.entities);
```

MapLibre GL JS — interactive vector maps and cartographic basemaps

MapLibre GL JS is a TypeScript library using WebGL to render **interactive maps from vector tiles**, with appearance controlled by a style specification. Current examples cover terrain, 3D buildings, clustering, heatmaps and integration with deck.gl. [36]

Its relationship to Cesium is complementary rather than strictly competitive. MapLibre is usually the simpler choice for conventional slippy maps, cartographic vector-tile applications and 2.5D city views. Cesium is stronger when you truly require a globe, ellipsoidal coordinates, terrain/3D Tiles and planet-scale 3D. For analytics-heavy interactive maps, **MapLibre + deck.gl** is one of the clearest modern combinations. [37]

```
import maplibregl from 'maplibre-gl';

const map = new maplibregl.Map({
  container: 'map',
  style: styleDocument,
  center: [151.21, -33.87],
  zoom: 10
});
```

OpenLayers remains an important nearby alternative, especially for broad 2D GIS/raster/vector/projection workflows; its current site is on v10.10.0. I would include it in a broader GIS survey, but for this particular graphics-oriented map, MapLibre is the more important addition because it sits directly beside deck.gl and modern GPU/vector-tile rendering. [38]

vtk.js — scientific, medical and engineering visualisation

vtk.js is explicitly a **scientific visualisation library for the web**, adapting the concepts and expertise of the VTK ecosystem to browser rendering. It has data models and rendering pipelines for polygonal and image/volume data, GPU volume rendering, transfer functions, interaction widgets and scientific visualisation semantics that would be substantial work to recreate on a generic 3D library. [39]

This makes vtk.js the natural choice for CT/MRI volume rendering, computational-fluid-dynamics fields, finite-element results, scalar/vector fields, meshes and other scientific datasets. Three.js can certainly display the resulting triangles, but vtk.js knows far more about *scientific data*. Its disadvantage is the converse: for a game or marketing experience, the VTK pipeline is specialist overhead. Release **36.6.2** appeared in August 2026 with continuing WebGPU-related changes, providing strong evidence of active maintenance. [40]

```
import vtkFullScreenRenderWindow
  from '@kitware/vtk.js/Rendering/Misc/FullScreenRenderWindow';
import vtkActor
  from '@kitware/vtk.js/Rendering/Core/Actor';
import vtkMapper
  from '@kitware/vtk.js/Rendering/Core/Mapper';
import vtkConeSource
  from '@kitware/vtk.js/Filters/Sources/ConeSource';

const view = vtkFullScreenRenderWindow.newInstance();
const cone = vtkConeSource.newInstance();
const mapper = vtkMapper.newInstance();

mapper.setInputConnection(cone.getOutputPort());

const actor = vtkActor.newInstance();
actor.setMapper(mapper);

view.getRenderer().addActor(actor);
view.getRenderer().resetCamera();
view.getRenderWindow().render();
```

xeokit — specialised high-performance BIM/AEC viewer SDK

xeokit is not trying to be a general game engine. It is an open-source WebGL toolkit designed around **large BIM and AEC models**, with IFC/BCF-oriented metadata, point clouds, engineering precision and specialised model loading. Its XKT format is designed as a compact, fast-loading representation for browser viewing; official conversion tooling supports workflows from IFC and other engineering formats. [41]

Its strengths become apparent with complex federated building models: engineering metadata, sectioning, measurement, object visibility/state, BCF-style workflows and precision are closer to native concepts than they would be in Three.js. The trade-offs are a more specialist ecosystem, a conversion pipeline when using XKT, and licensing considerations: the open SDK uses AGPL licensing with commercial options available. The project remained active through 2026, with the current v2.6 series continuing to receive releases. [42]

```
import {
  Viewer,
  XKTLoaderPlugin
} from '@xeokit/xeokit-sdk';

const viewer = new Viewer({
  canvasId: 'xeokitCanvas'
});

const loader = new XKTLoaderPlugin(viewer);

loader.load({
  id: 'building',
  src: 'building.xkt'
});
```

The choice between xeokit and Three.js is therefore analogous to vtk.js versus Three.js: choose Three when you want the general rendering substrate; choose xeokit when your application is intrinsically **BIM/AEC** and its domain features outweigh the value of a neutral engine. [43]

That Open Components / Fragments / web-ifc — an IFC-native BIM application stack

That Open is increasingly better thought of as a **stack** rather than one library. `web-ifc` is the low-level WASM-backed IFC reader/writer. The Fragments system provides an optimised model representation and geometry pipeline. `@thatopen/components` supplies higher-level BIM application components built around Three.js, including worlds/renderers/cameras, clipping, measurement and floor-plan/navigation functionality. Official Components documentation explicitly builds its minimal world using Three.js-based scene infrastructure. [44]

This is a particularly attractive architecture when you want **open IFC ingestion plus a custom Three.js-based BIM product**. Compared with xeokit, it feels less like adopting a dedicated monolithic viewer SDK and more like assembling a BIM application from modular pieces. That flexibility brings more moving parts: Three.js concepts, IFC semantics, WASM setup, worker/fragments lifecycle and relatively fast-moving APIs all become part of the engineering surface. Components reached the 3.4 line in April 2026, and web-ifc development remained active during 2026. [45]

Minimal Components world:

```
import * as OBC from '@thatopen/components';

const components = new OBC.Components();
const worlds = components.get(OBC.Worlds);
```

```
const world = worlds.create();

world.scene = new OBC.SimpleScene(components);
world.renderer = new OBC.SimpleRenderer(
  components,
  document.getElementById('app')
);
world.camera = new OBC.SimpleCamera(components);

components.init();
world.scene.setup();
```

At the lower level, web-ifc can be used directly when you need IFC data rather than the whole viewing stack:

```
import { IfcAPI } from 'web-ifc';

const ifc = new IfcAPI();
await ifc.Init();

const bytes = new Uint8Array(await file.arrayBuffer());
const modelID = ifc.OpenModel(bytes);

// Query properties/geometry through the IfcAPI...
```

The official web-ifc quick start follows exactly this model: initialise `IfcAPI`, open IFC bytes, query the resulting model and close it when finished. [46]

For new BIM products, a useful distinction is:

```
xeokit
  → specialist BIM/AEC viewer SDK
  → strongly optimised engineering viewing workflow
  → XKT-centric performance pipeline available

That Open
  → modular BIM components around Three.js
  → direct IFC → Fragments workflow
  → attractive when custom product architecture matters

web-ifc
  → IFC parser/writer + geometry engine layer
  → not by itself a complete viewer
```

That distinction follows their respective official architectures. [47]

OpenCascade.js — genuine CAD geometry kernel in WebAssembly

OpenCascade.js differs more profoundly from the other libraries in this report than its name suggests. It is a JavaScript/WebAssembly binding of **Open CASCADE Technology**, the established C++ CAD/CAM/CAE geometry platform. It can construct topological solids, curves and surfaces, perform Boolean operations and participate in STEP/IGES-style CAD pipelines. The official documentation currently still directs users to the beta package line, so it should be regarded as a more specialised and less frictionless dependency than mature renderers such as Three.js. [48]

The crucial conceptual distinction is **B-rep versus triangles**. A Three.js mesh of a cylinder is principally something to render. An OpenCascade cylinder can participate in exact topological and geometric operations: faces, edges, Booleans, fillets and CAD interchange. You normally still need a renderer—often by tessellating/exporting the CAD result and displaying it through Three.js or another WebGL/WebGPU system. Official OpenCascade.js examples explicitly triangulate CAD data and export GLB for rendering. [49]

```
import initOpenCascade from 'opencascade.js';

const oc = await initOpenCascade();

const box = new oc.BRepPrimAPI_MakeBox_2(10, 20, 30);
const sphere = new oc.BRepPrimAPI_MakeSphere_5(
  new oc.gp_Pnt_3(5, 10, 15),
  8
);

const cut = new oc.BRepAlgoAPI_Cut_3(
  box.Shape(),
  sphere.Shape(),
  new oc.Message_ProgressRange_1()
);

cut.Build(new oc.Message_ProgressRange_1());

const cadShape = cut.Shape();
```

Its disadvantages are predictable for a WebAssembly CAD kernel: binary size, generated API ergonomics, specialist CAD knowledge and heavier computations. The documentation notes that visualisation itself requires meshing/export steps and that bundler configuration is part of setup. Nevertheless, for a browser **CAD editor**, ignoring OpenCascade.js while comparing only Three.js, Konva or Fabric would conflate “drawing geometry” with “modelling CAD geometry”. [50]

Scenario-by-scenario recommendations

The following recommendations are inferences from the libraries' current architecture, official feature sets and maintenance state rather than vendor-provided rankings.

Scenario	Strong default	Common companions	Why
2D game	**Phaser**	Matter.js, GSAP	Game lifecycle, scenes, sprites, input and tilemaps already exist. [51]
Custom/high-throughput 2D renderer	**PixiJS**	GSAP, React/ECS	Lower-level and less opinionated than Phaser; WebGL/WebGPU scene renderer. [52]
3D browser game	**Babylon.js or PlayCanvas**	Rapier/custom physics where appropriate	More engine-level systems than bare Three.js. [53]
Highly bespoke 3D application	**Three.js**	Rapier, GSAP, custom UI	Gives the least opinionated general 3D substrate. [54]
Marketing / interactive animation	**GSAP + Three.js/ PixiJS/DOM**	ScrollTrigger and renderer-specific integrations	GSAP handles choreography while the renderer handles pixels. [17]
Whiteboard / diagram / floor-plan editor	**Konva**	React, custom snapping/ geometry	Strong interaction, dragging, transforms and scene graph. [55]
Graphic/design editor	**Fabric.js**	Custom application state	Rich editable objects, text, SVG and serialisation. [20]
Precision 2D CAD-like editor	**Konva/Fabric + geometry layer**	CAD/constraint code	Canvas libraries handle interaction, but precise CAD semantics remain separate. [56]
True 3D CAD modeller	**OpenCascade.js + Three.js**	custom topology/ selection UI	OpenCascade supplies CAD B-rep operations; Three renders the tessellation. [49]
Architectural walkthrough without deep BIM semantics	**Three.js, Babylon.js or PlayCanvas**	GSAP / physics	Good general real-time 3D choices for glTF/GLB environments. [57]
AEC/BIM viewer or coordination app	**xeokit or That Open**	IFC/BCF backend, Three.js in That Open stack	Domain-specific metadata, IFC and building interaction features matter. [58]
Simple 2D rigid-body physics	**Matter.js**	Phaser/PixiJS	Easy pure-JS 2D simulation. [59]

Scenario	Strong default	Common companions	Why
Performance-sensitive 2D/3D physics	**Rapier**	Three.js, PixiJS, custom engine	WASM, 2D/3D and deterministic JS/WASM simulation. [60]
Standard scientific charts	**Plotly.js**	D3 for custom extras	High-level scientific, statistical and 3D traces. [61]
Bespoke data visualisation	**D3**	Canvas/SVG/React	Maximum control over scales, shapes and data joins. [62]
Large GPU analytical datasets	**deck.gl**	MapLibre GL JS	GPU layer architecture; natural map integration. [33]
Medical / volume / CFD / FEA visualisation	**vtk.js**	Plotly/D3 for charts	Scientific data pipelines and GPU volume rendering are native concerns. [63]
Normal interactive web map	**MapLibre GL JS**	deck.gl, Turf	Vector-tile/cartographic model fits directly. [64]
Globe, terrain, 3D Tiles, planet-scale digital twin	**CesiumJS**	deck.gl/custom overlays	Designed around precision 3D geospatial data. [34]

A few choices deserve more nuance.

For **2D games**, choose Phaser unless you have a concrete reason to own more infrastructure yourself. PixiJS becomes preferable when the application is game-like visually but structurally behaves more like a custom web product, visual editor or very high-throughput display. Matter.js can sit underneath either when lightweight physical behaviour is needed. [65]

For **3D games**, Babylon.js is probably the most straightforward code-first default from this set when you want a full engine; PlayCanvas is especially compelling when visual tooling and ECS workflow are central. Choose Three.js when the product's architecture matters more than having conventional engine systems prebuilt. This is not a quality ranking; it is primarily a question of abstraction level. [66]

For **floor plans**, the word “CAD” determines the answer. A draggable browser floor-plan designer with rooms, furniture and labels can be very comfortable in Konva or Fabric. A BIM floor-plan view that must correspond to IFC objects belongs closer to That Open or xeokit. A genuine parametric/geometric CAD editor requiring exact intersections, B-rep topology or STEP exchange needs a geometric kernel such as OpenCascade.js beneath its UI. [67]

For **architectural walkthroughs**, BIM-aware tools are not automatically better. If the deliverable is an attractive interactive walkthrough from pre-exported GLB/glTF assets, Three.js, Babylon or PlayCanvas generally gives a more natural real-time graphics stack. If the user must query `IfcWall`, property sets, storeys, systems, BCF issues and federated engineering metadata, xeokit or That Open becomes materially more appropriate. [68]

For **scientific simulation**, distinguish the solver from the visualiser. Rapier is a rigid-body physics solver, not a general PDE/FEA/CFD solver. vtk.js is a scientific **visualisation** toolkit, not the numerical simulation engine. Plotly and D3 visualise derived results. More sophisticated browser simulation systems often execute domain-specific numerical code in WebAssembly or WebGPU and use one of these libraries only to display the results. Babylon and Three can expose WebGPU capabilities, but that does not turn them into general scientific solvers. [69]

For **BIM**, I would put xeokit and That Open on the shortlist before starting a fresh bespoke layer directly over Three.js. Reimplementing object metadata, high-performance model partitioning, IFC loading, clipping, measurements, storey navigation and engineering selection is a large amount of non-rendering work. Direct Three.js remains reasonable where requirements are narrow or where full control justifies that engineering cost. [70]

For **geospatial applications**, the most useful boundary is: MapLibre for the map, deck.gl for large analytical layers, Cesium for a genuinely three-dimensional Earth. A project may contain more than one of them rather than choosing one universally. deck.gl officially supports MapLibre integration, while Cesium's native model starts from globe-scale 3D geospatial visualisation. [37]

Practical ecosystem mental map

The most useful mental model is to separate **renderers**, **application/game frameworks**, **animation**, **physics**, **domain visualisation** and **geometry kernels**:

```
Browser graphics / simulation ecosystem
│
├── General 3D rendering
│   │
│   ├── Three.js
│   │   ├── low/mid-level general 3D scene + renderer
│   │   ├── commonly + Rapier       → 3D physics
│   │   ├── commonly + GSAP         → animation/choreography
│   │   ├── underneath That Open    → BIM
│   │   └── + OpenCascade.js        → browser CAD
│   │
│   ├── Babylon.js
│   │   └── higher-level integrated 3D/game engine
│   │
│   └── PlayCanvas
│       └── 3D engine + ECS + editor-oriented workflow
│
├── General 2D rendering / games
│   │
│   ├── PixiJS
│   │   ├── fast 2D WebGL/WebGPU renderer
│   │   ├── + GSAP                  → rich animated experiences
│   │   └── + Matter/Rapier         → physics
│   │
│   └── Phaser
│       ├── full 2D game framework
```

```
│       └── + Matter.js / own systems
│
├── Animation
│   │
│   └── GSAP
│       ├── DOM / CSS / SVG
│       ├── + Three.js
│       ├── + PixiJS
│       └── + custom JavaScript objects
│
├── Interactive 2D editors
│   │
│   ├── Konva
│   │   └── diagrams / whiteboards / floor plans
│   │
│   └── Fabric.js
│       └── graphic editors / text / SVG / serialised documents
│
├── Physics
│   │
│   ├── Matter.js
│   │   └── accessible pure-JS 2D rigid bodies
│   │
│   └── Rapier
│       └── WASM 2D + 3D rigid-body physics
│
├── Data visualisation
│   │
│   ├── D3.js
│   │   └── low-level bespoke visual encodings
│   │
│   ├── Plotly.js
│   │   └── high-level charts / scientific plots / 3D plots
│   │
│   └── deck.gl
│       ├── GPU visualisation for huge datasets
│       └── commonly + MapLibre
│
├── Geospatial
│   │
│   ├── MapLibre GL JS
│   │   └── vector-tile maps / cartography / 2.5D
│   │
│   └── CesiumJS
│       └── globe / terrain / 3D Tiles / planet-scale 3D
│
├── Scientific / engineering visualisation
│   │
│   └── vtk.js
│       └── meshes / volume data / medical / CFD / FEA results
```

```
│
├── BIM / AEC
│   │
│   ├── xeokit
│   │   └── specialist high-performance BIM/AEC SDK
│   │
│   └── That Open stack
│       ├── web-ifc              → IFC parsing / writing / geometry
│       ├── Fragments            → optimised BIM model representation
│       ├── Components           → BIM application features
│       └── Three.js             → underlying 3D ecosystem
│
└── True CAD geometry
    │
    └── OpenCascade.js
        ├── B-rep / curves / surfaces / topology
        ├── Boolean modelling
        ├── CAD interchange workflows
        └── usually + Three.js or another renderer
```

That map captures the central architectural lesson: **do not choose a renderer to solve a domain-model problem**. Three.js can render an engineering mesh, but vtk.js understands scientific visualisation. Three.js can render a building, but xeokit and That Open understand BIM. Canvas can draw a floor plan, but Fabric and Konva understand editable graphical objects. Three.js can display tessellated STEP geometry, but OpenCascade.js provides the CAD topology and geometry operations. [71]

Likewise, **do not choose a full engine when a renderer is all you need**. A custom product visualiser may be simpler and easier to control with Three.js; a complete 3D browser game may benefit from Babylon or PlayCanvas. A 2D interactive visualisation may need PixiJS, while a conventional game usually benefits from Phaser's game-specific lifecycle. [72]

The combinations that make the most architectural sense in 2026 are therefore:

```
Three.js     + Rapier          → bespoke 3D + serious physics
Three.js     + GSAP            → polished interactive 3D
Three.js     + OpenCascade.js  → web CAD
Three.js     + That Open       → custom BIM products

PixiJS       + GSAP            → high-end interactive 2D animation
PixiJS       + Matter/Rapier   → custom 2D game/simulation architecture
Phaser       + built-ins       → conventional browser 2D game

Konva        + React           → diagram / plan / annotation editor
Fabric.js    + app state       → graphic/design editor

D3           + SVG/Canvas      → bespoke visualisation
Plotly.js    + scientific data → standard analytical/scientific charts
MapLibre     + deck.gl         → large-scale interactive map analytics
```

```
vtk.js       + Plotly/D3       → engineering/scientific 3D + charts
xeokit       + BIM backend     → large specialist AEC/BIM viewer
CesiumJS     + 3D Tiles        → globe-scale geospatial/digital twin
```

These combinations follow the libraries' deliberate separation of responsibilities: GSAP animates arbitrary properties, Rapier owns physics, deck.gl officially integrates with MapLibre, That Open builds its BIM component stack around Three.js, vtk.js supplies the scientific rendering pipeline, and OpenCascade.js can generate/triangulate CAD geometry for downstream renderers. [73]

The resulting high-level selection rule is simple. For **games**, start with Phaser for 2D and Babylon/PlayCanvas for integrated 3D, dropping to PixiJS or Three.js when custom architecture is a priority. For **interactive motion**, add GSAP rather than changing renderers. For **editors**, start with Konva or Fabric unless true CAD/BIM semantics change the problem. For **physics**, use Matter for straightforward 2D and Rapier for demanding 2D/3D. For **data**, move from Plotly at the high level to D3 for bespoke graphics and deck.gl for massive GPU datasets. For **scientific/engineering visualisation**, vtk.js is much closer to the domain than a generic renderer. For **BIM**, shortlist xeokit and That Open. For **CAD**, use a real kernel such as OpenCascade.js. For **geospatial**, think MapLibre → deck.gl → Cesium as increasingly specialised steps from cartographic mapping through analytical GPU layers to globe-scale 3D. [74]

---

[1] [5] [6] [11] [27] [54] [57] [72] Three.js – JavaScript 3D Library
https://threejs.org/?utm_source=chatgpt.com

[2] Concepts | PlayCanvas Developer Site
https://developer.playcanvas.com/user-manual/react/guide/?utm_source=chatgpt.com

[3] [14] Download Phaser v4.2.1
https://phaser.io/download/release/v4.2.1?utm_source=chatgpt.com

[4] pixi.js
https://www.npmjs.com/package/pixi.js?activeTab=readme&utm_source=chatgpt.com

[7] [53] [66] Babylon.js: Powerful, Beautiful, Simple, Open - Web-Based 3D ...
https://www.babylonjs.com/?utm_source=chatgpt.com

[8] New: Large World Rendering! - Announcements
https://forum.babylonjs.com/t/new-large-world-rendering/61114?utm_source=chatgpt.com

[9] PlayCanvas | Open Source WebGL & WebGPU Game Engine
https://playcanvas.com/?utm_source=chatgpt.com

[10] Releases · playcanvas/engine
https://github.com/playcanvas/engine/releases?utm_source=chatgpt.com

[12] PixiJS | The HTML5 Creation Engine | PixiJS
https://pixijs.com/?utm_source=chatgpt.com

[13] Graphics Fill
https://pixijs.com/8.x/guides/components/scene-objects/graphics/graphics-fill?utm_source=chatgpt.com

[15] Getting Started with Phaser 3 - Part 1 - Introduction
https://phaser.io/tutorials/getting-started-phaser3?utm_source=chatgpt.com

[16] [73] gsap.to() | GSAP | Docs & Learning
https://gsap.com/docs/v3/GSAP/gsap.to%28%29/?utm_source=chatgpt.com

17 Timeline | GSAP | Docs & Learning
https://gsap.com/docs/v3/GSAP/Timeline/?utm_source=chatgpt.com

18 55 56 Getting Started with Konva — HTML5 Canvas 2D Framework
https://konvajs.org/docs/index.html?utm_source=chatgpt.com

19 HTML5 Canvas Drag and Drop Tutorial
https://konvajs.org/docs/drag_and_drop/Drag_and_Drop.html?utm_source=chatgpt.com

20 21 Fabric.js Javascript Library
https://fabricjs.com/?utm_source=chatgpt.com

22 52 Introduction | PixiJS
https://pixijs.com/8.x/guides/getting-started/intro?utm_source=chatgpt.com

23 24 59 Matter.js - a 2D rigid body JavaScript physics engine - brm·io
https://brm.io/matter-js/?utm_source=chatgpt.com

25 60 Determinism
https://rapier.rs/docs/user_guides/javascript/determinism/?utm_source=chatgpt.com

26 GitHub - dimforge/rapier.js: Official JavaScript bindings for ...
https://github.com/dimforge/rapier.js/?utm_source=chatgpt.com

28 D3 by Observable | The JavaScript library for bespoke data ...
https://d3js.org/?utm_source=chatgpt.com

29 Reference index in JavaScript
https://plotly.com/javascript/reference/index/?utm_source=chatgpt.com

30 61 Plotly JavaScript Open Source Graphing Library
https://plotly.com/javascript/?utm_source=chatgpt.com

31 Issues · plotly/plotly.js
https://github.com/plotly/plotly.js/issues?utm_source=chatgpt.com

32 33 Introduction | deck.gl
https://deck.gl/docs?utm_source=chatgpt.com

34 CesiumJS
https://cesium.com/platform/cesiumjs/?utm_source=chatgpt.com

35 Releases · CesiumGS/cesium
https://github.com/CesiumGS/cesium/releases?utm_source=chatgpt.com

36 64 Introduction - MapLibre GL JS
https://www.maplibre.org/maplibre-gl-js/docs/?utm_source=chatgpt.com

37 Using with MapLibre
https://deck.gl/docs/developer-guide/base-maps/using-with-maplibre?utm_source=chatgpt.com

38 OpenLayers - Welcome
https://openlayers.org/?utm_source=chatgpt.com

39 63 71 Overview | VTK.js
https://kitware.github.io/vtk-js/docs/?utm_source=chatgpt.com

40 Releases · Kitware/vtk-js
https://github.com/kitware/vtk-js/releases?utm_source=chatgpt.com

41 43 47 58 70 Build Faster 3D Web Apps for BIM | Open-Source xeokit SDK
https://xeokit.io/?utm_source=chatgpt.com

42 GitHub - xeokit/xeokit-sdk: 3D BIM IFC Viewer SDK for AEC engineering applications. Open Source JavaScript Toolkit based on pure WebGL for top performance, real-world coordinates and full double precision · GitHub
https://github.com/xeokit/xeokit-sdk

44 46 That Open Engine | web-ifc
https://thatopen.github.io/engine_web-ifc/docs/?utm_source=chatgpt.com

45 Releases · ThatOpen/engine_components
https://github.com/ThatOpen/engine_components/releases?utm_source=chatgpt.com

48 OpenCascade.js | Open CASCADE Technology
https://dev.opencascade.org/project/opencascadejs?utm_source=chatgpt.com

49 50 Hello, World! | OpenCascade.js
https://ocjs.org/docs/getting-started/hello-world

51 65 74 Welcome to Phaser Docs | Phaser Help
https://docs.phaser.io/?utm_source=chatgpt.com

62 What is D3? | D3 by Observable
https://d3js.org/what-is-d3?utm_source=chatgpt.com

67 Why Konva? When to Use Konva.js for Your Project
https://konvajs.org/docs/guides/why-konva.html?utm_source=chatgpt.com

68 | PlayCanvas Developer Site
https://developer.playcanvas.com/user-manual/react/api/gltf/?utm_source=chatgpt.com

69 Getting started | Rapier
https://rapier.rs/docs/user_guides/javascript/getting_started_js/?utm_source=chatgpt.com